Task 11 – Recommend and Defend the Final Design

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K5 – Evaluate

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task11)

1. Recommended design

Decision	Recommendation	Evidence from my experiments
Type of money	**Decimal** built from the text the clerk typed (or integer paise); never float	Task 2: float gave 33400.30000000061 and Decimal(float(...)) kept the error; Task 10: 7,324 wrong paise (Rs 73.24) in 150,000 float payments.
Rounding rule	Round **each payment** to the paisa, **half up**, with Decimal; add the rounded amounts	Task 10: rounding only the total was Rs 72.14 away from the sum of paid amounts; Task 9 test B5 (1.505 → 1.51).
Step counting	round() to the nearest step on exact Decimal values	Task 4: int() lost a step for fat 3.1 and both int() and floor() lost a step for fat 4.1 (Rs 0.40/L each); round() was right in every case.
Structure	Constants in one rate card; small functions (rate_per_litre, payout, ten_day_total) in a module; validation before calculation	Task 3: one edit per revision; Task 6: function overhead ≈ 10 %, negligible; Task 9: 18 of 18 tests pass.
Speed-up	Optional rate table of (fat, snf) pairs	Task 10: 886 → 187 ns per entry (float), 2,408 → 184 ns (Decimal).
Memory at scale	Running total per farmer; do not keep all entries unless needed	Task 8: O(1) versus 16.5 MB at n = 150,000.

2. How a rate revision will be carried out

- Edit only the rate card (BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE) – one place, no other file.
- Run test_milkcalc.py; the expected values of the rate-dependent tests are recalculated by hand and updated at the same time. All other tests (validation, rounding, hand total) must still pass.
- Commit the change with the date and the union's order number; run one old entry through both old and new rate to confirm the difference equals the revision.
- If a rate table (Task 10) is used, rebuild it at start-up so it can never be stale.

3. Limitations of the design

- Readings are assumed to have one decimal; the two-decimal analyser needs the pro-rata step rule (Task 12).
- Only one rate card at a time and no bonus rules; seasonal rates need a rate card per season (Task 12).
- SNF limits (6.0–10.0) and the 200 L entry cap are my assumptions and must be confirmed with the union.
- Decimal is about two to three times slower than float (Task 10) and needs the rule "never build Decimal from float" to be followed by every programmer.
- A text rate card must be validated; a typing slip in the file (0.04 for 0.40) would be paid silently unless range checks exist.
- The program calculates payments but does not store them; a database and audit trail are outside this assignment.

4. Is the design correct at district-union scale?

Correctness: yes – Decimal arithmetic is exact at any n, the rounding rule is defined per payment, and the 18 tests do not depend on n. **Speed:** measured ≈ 3.5 µs per payment with Decimal. A district union has 150,000 farmers; at an assumed 20 entries per farmer per ten days that is 3 million

entries, i.e. roughly 10 seconds (an estimate from my measurement) – acceptable for a job run every ten days, and the rate table reduces it further. **Memory:** O(1) with running totals; O(n) only if all entries are kept (16.5 MB per 150,000 entries measured).

Alternative Concept and Complexity Analysis

Alternative: read the rate card from a small JSON file when the program starts, instead of writing it in the module.

ratecard.json and load_card.py (excerpt)

```
{
  "base_rate": "30.00",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30",
  "std_fat": "3.5",
  "std_snf": "8.5",
  "step": "0.1"
}

import json, time, timeit
from decimal import Decimal

def load_card(path="ratecard.json"):
    with open(path) as fh:
        raw = json.load(fh)
    required = {"base_rate", "fat_step_value", "snf_step_value", "std_fat", "std_snf", "step"}
    missing = required - raw.keys()
    if missing:
        raise ValueError(f"rate card is missing: {sorted(missing)}")
    return {k: Decimal(v) for k, v in raw.items()}
```

Measured (load_card.py)

```
{'base_rate': '30.00', 'fat_step_value': '0.40', 'snf_step_value': '0.30', 'std_fat': '3.5', 'std_snf': '8.5', 'step':
start-up cost to read 6 values: 20 us
memory of the card (dict + values): 896 bytes
```

	Rate card in the module	Rate card in a JSON file
Start-up time (t = 6 values)	none (values compiled in)	≈ 20 µs: O(t) to read and parse; irrelevant beside Python start-up
Space	t constants	O(t): about 900 bytes for t = 6 Decimal values
After a revision	edit the .py file, retest, redistribute the program to every collection centre	edit one small text file; the program is not touched or redistributed
Risk	a code edit may break something else	a bad value or missing key; mitigated by validation (the loader checks the required keys)

Reading six values costs about 20 microseconds once per run – negligible – while redistributing code to every centre after each revision costs people time and risks version mismatch. The JSON file is therefore recommended *for the values that change* (rate card, season dates), with the validation shown, and the calculation code stays in the module.

Evidence

A recommendation supported by the measurements of Tasks 2, 4, 6, 8, 9 and 10 and the JSON loading test. Source: Task11/ratecard.json, load_card.py.